Since 1983

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

PEER TO PEER FILE DISTRIBUTION SYSTEM

Mini-project presentation

Computer Networks Laboratory (CSE20110)
Department of Computer Engineering and Technology

Rachel Tom (1262240226) Manasi Sharma (1262240248)
Aaron Johnson (1262240581) Aaron Antao (1262240595)

Group No. 5 Panel K Batch 1
Project guide: Dr Uma Pujeri
Academic Year: 2026–2027

Problem and Objectives

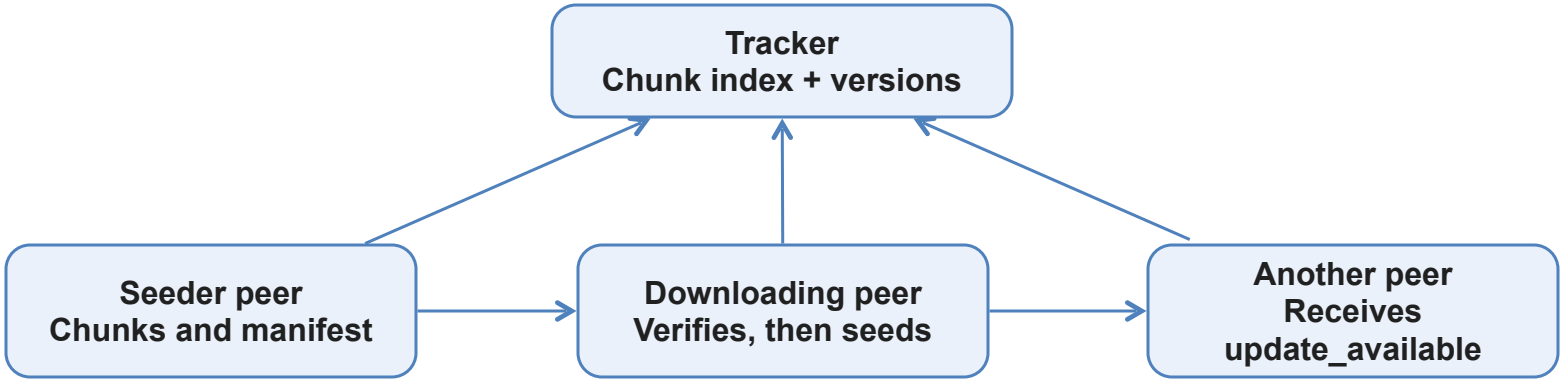
Problem

A single file server is a bandwidth bottleneck and a single point of failure, and every update has to be re-sent to every user. Peers should share verified chunks with each other and stay current when the file changes.

Objectives

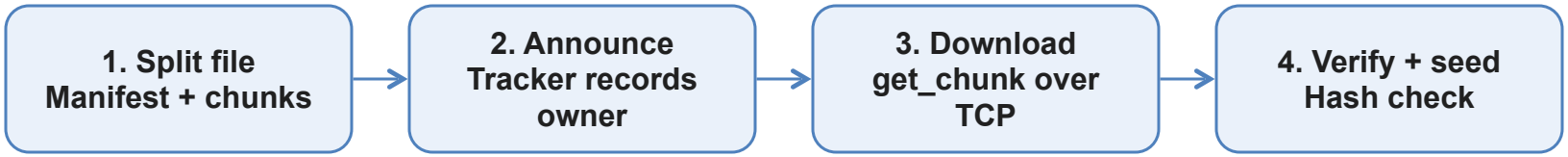
1. Split a file into chunks and record SHA-256 digests in a manifest.
2. Retrieve, verify and seed chunks through direct TCP peer connections.
3. Rebuild the original file with an identical SHA-256 digest.
4. Notify every peer of a new version and deliver it through the peers.

Proposed Design and Tools



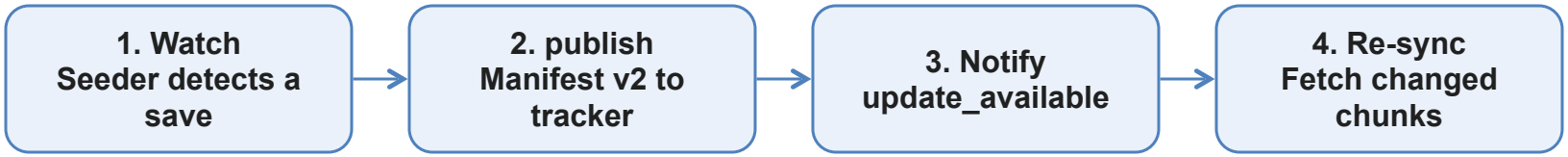
Python 3.10+ standard library only; no third-party packages
TCP sockets carrying newline-delimited JSON messages
Tracker 127.0.0.1:9000 · seeder 9101 · downloaders 9102–9104
The tracker never carries file data; chunks flow peer to peer

Implementation and Working



Message	Direction	Purpose
announce	Peer → tracker	Register owned chunks; reply carries latest version
lookup	Peer → tracker	Return the owners of each chunk
get_chunk	Peer → peer	Return one chunk's data (Base64)
publish	Seeder → tracker	Record a new manifest version
get_manifest	Peer → tracker	Fetch the newest manifest
update_available	Tracker → peer	Push notice that a newer version exists

File Update Notification



Consistency safeguards

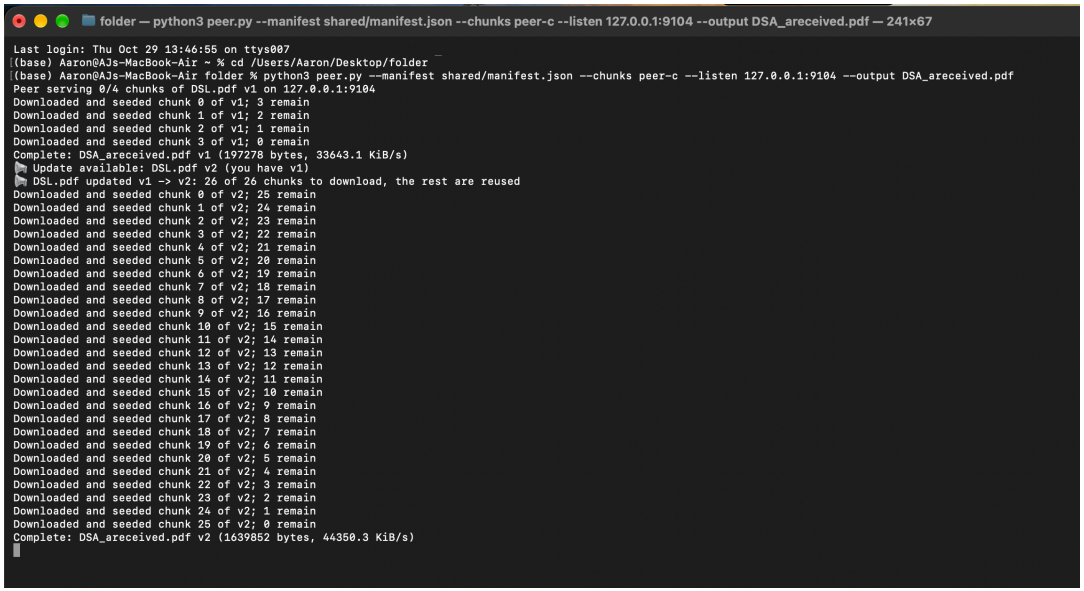
Chunks are stored as <sha256>.bin, so versions never overwrite each other
Unchanged chunks are reused; only changed chunks are downloaded
Whole-file SHA-256 is checked; output is replaced in one atomic step
A peer that misses the push learns of v2 on its next announce (≤ 25 s)

Testing and Results

ID	Test	Observed result	Status
T1	Create manifest from DSL.pdf	26 chunks and manifest.json created	Pass
T2	Download version 1	4 of 4 chunks downloaded and seeded	Pass
T3	Publish version 2 while peer runs	“Update available: DSL.pdf v2” printed	Pass
T4	Download the new version	26 of 26 chunks; 1,639,852 bytes rebuilt	Pass
T5	Compare SHA-256 digests	Source and output digests identical	Pass
T6	Start peer with old v1 manifest	Peer switched to v2; digest matched	Pass
T7	Corrupt a chunk on the seeder	Rejected by SHA-256 check; not saved	Pass

SHA-256 (DSL.pdf) = SHA-256 (DSA_areceived.pdf) = 494dd4ff…eaaee3395b. Rates were measured on loopback, so no network throughput claim is made.

Execution Evidence

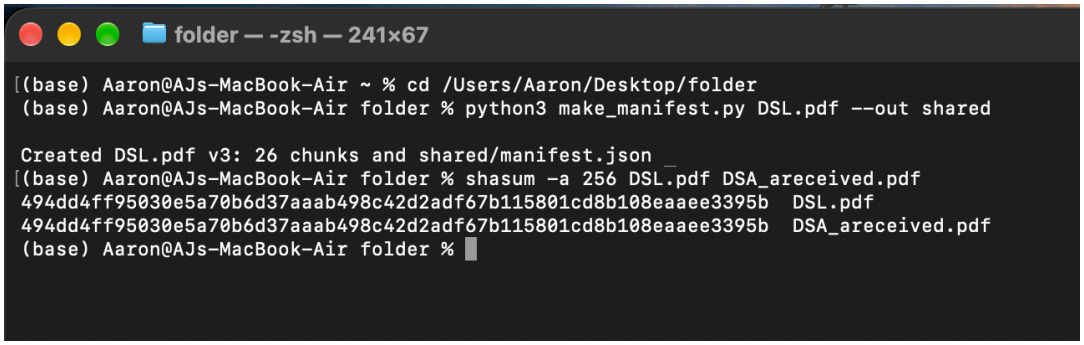


Downloading peer log

v1: 4 chunks downloaded and seeded

Update notice for v2 received

v2: 26 chunks fetched, file rebuilt



Integrity check

Source and rebuilt file have identical SHA-256 digests

Working Demonstration

Function to demonstrate

Chunk distribution, immediate seeding, a live file update and output verification.

1. Start tracker.py and run make_manifest.py DSL.pdf --out shared.
2. Start the seeder on port 9101 with --seed --watch DSL.pdf.
3. Start a downloader on port 9104; show “Downloaded and seeded” lines.
4. Save a new DSL.pdf; show the update notice and re-download.
5. Compare SHA-256 digests of DSL.pdf and DSA_areceived.pdf.

Success criterion: both files have the same SHA-256 digest after the update.

Conclusion and Contributions

Outcome: working TCP peer-to-peer distribution with version updates
Application: LAN sharing of course files, software images, datasets
Limitation: single tracker, Base64 payloads, unsigned manifests
Future scope: parallel downloads, TLS, signed manifests, DHT

Member	Contribution
Rachel Tom	Testing and execution verification
Manasi Sharma	Chunking and integrity verification (make_manifest.py, peer.py)
Aaron Johnson	Report and presentation
Aaron Antao	Tracker and protocol design (tracker.py, protocol.py)

AI use: code scaffolding and report drafting; the group verified all code and results.

References

[1] B. Cohen, “The BitTorrent Protocol Specification,” 2008.
https://www.bittorrent.org/beps/bep_0003.html

[2] J. Postel, “Transmission Control Protocol,” RFC 793, 1981.
https://doi.org/10.17487/RFC0793

[3] NIST, “Secure Hash Standard,” FIPS PUB 180-4, 2015.
https://doi.org/10.6028/NIST.FIPS.180-4

Project source: tracker.py, peer.py, make_manifest.py, protocol.py
github.com/aaronantao/p2p-file-distribution